

Lab 4 - Logic Operations

June 7, 2025

1 Theory

Logical operators are fundamental tools in digital image processing, enabling pixel-wise manipulation of images. The most common logical operations include AND, OR, and XOR, which are applied to corresponding pixels of two images. These operations are particularly useful for tasks such as masking, image blending, and feature extraction.

Let $A(x, y)$ and $B(x, y)$ represent the pixel values at position (x, y) in two grayscale images.

- **AND Operation:**

$$C(x, y) = A(x, y) \wedge B(x, y)$$

Produces a result where a pixel is set only if the corresponding pixels in both input images are set. This is useful for extracting common features between two images.

- **OR Operation:**

$$C(x, y) = A(x, y) \vee B(x, y)$$

Produces a result where a pixel is set if the corresponding pixel in either input image is set. This operation is useful for combining features from both images.

- **XOR Operation:**

$$C(x, y) = A(x, y) \oplus B(x, y)$$

Produces a result where a pixel is set if the corresponding pixels in the input images are different. This highlights differences between two images.

Before applying these operations, images are typically converted to grayscale and resized to ensure they have the same dimensions. The operations are performed using NumPy arrays for efficient computation, and the results are converted back to image format for visualization.

```
[1]: #import libraries
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
```

```
[2]: #read the image
# reading image and converting to gray scale
img = Image.open('img/prakash.jpg').convert('L')
# display image
```

```
img
```

[2]:



```
[3]: img = img.resize((400,400), Image.Resampling.LANCZOS)
      # convert to numpy array
      numpy_image = np.array(img)
```

```
[4]: #read the image
      # reading image and converting to gray scale
      img2 = Image.open('img/tiger.png').convert('L')
      # display image
      img2
```

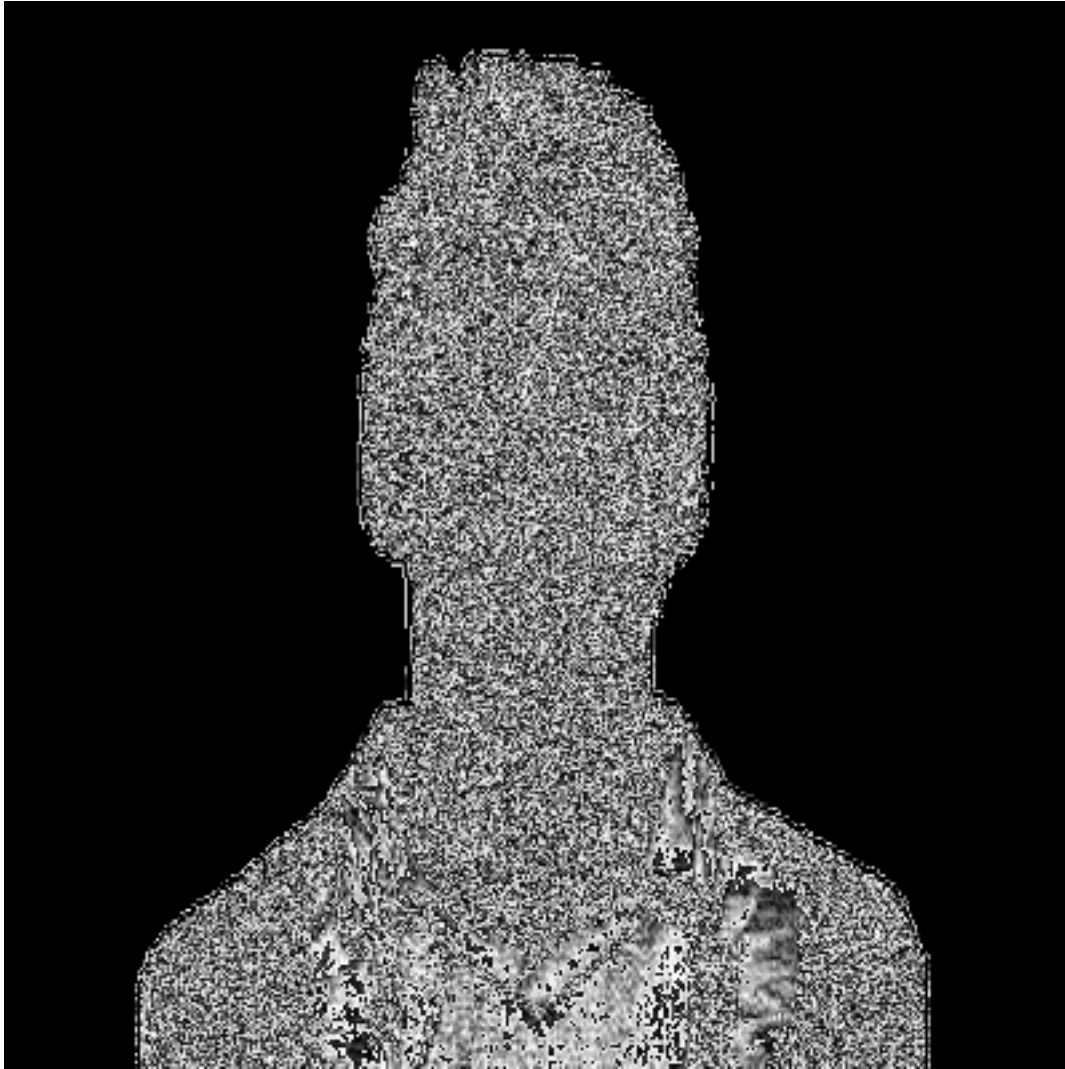
[4]:



```
[5]: img2 = img2.resize((400,400), Image.Resampling.LANCZOS)
     # convert to numpy array
     numpy_image2 = np.array(img2)
```

```
[6]: # Logic AND
     aa = numpy_image*numpy_image2
     aa = Image.fromarray(aa)
     aa = aa.convert("L")
     aa
```

```
[6]:
```



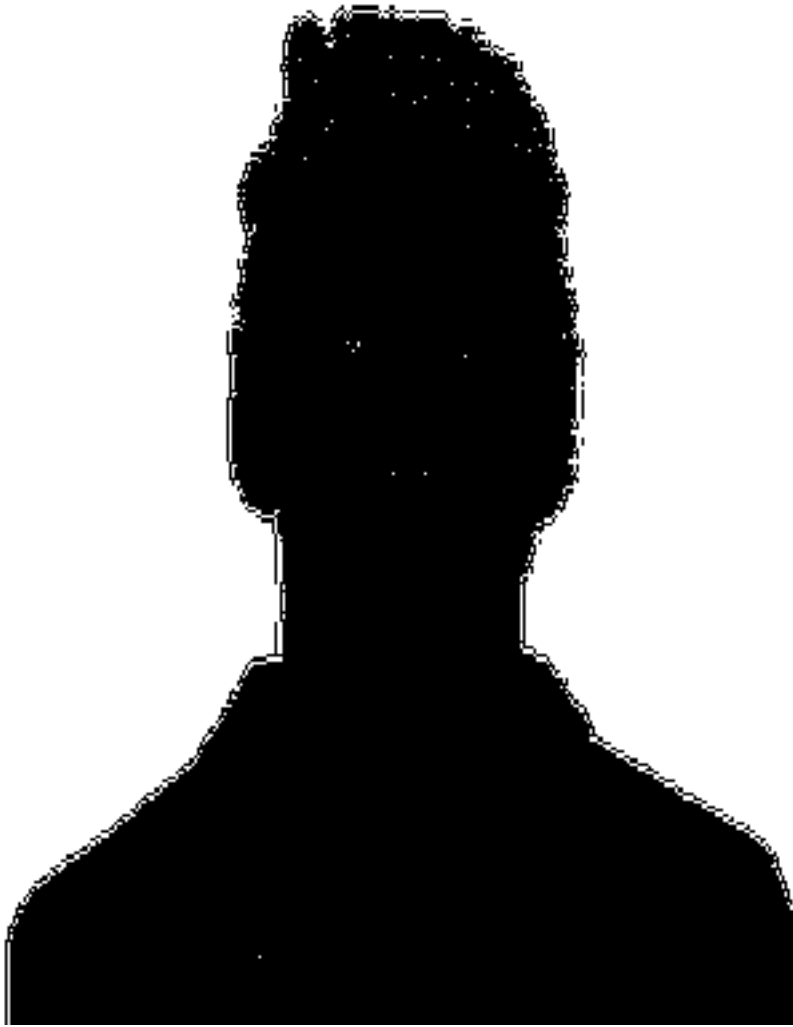
```
[7]: # Logic OR
aa = numpy_image+numpy_image2
aa = Image.fromarray(aa)
aa = aa.convert("L")
aa
```

[7]:



```
[8]: #XOR  
aa = np.logical_xor(numpy_image, numpy_image2)  
aa = Image.fromarray(aa)  
aa = aa.convert("L")  
aa
```

[8]:



2 Conclusion

In this lab, logical operators (AND, OR, XOR) were successfully applied to two grayscale images. The results demonstrated how each operator affects the combination of image features:

- The **AND** operation highlighted only the overlapping regions of both images.
- The **OR** operation combined the features of both images, showing all regions present in either image.
- The **XOR** operation emphasized the differences between the two images.

These logical operations are essential in various image processing applications, such as segmentation, masking, and change detection. The practical implementation in this notebook reinforces the importance and utility of logical operators in digital image analysis.